

NAME: AKSHAYA A

REG NO: 192421222

COURSE: CSA0603 DESIGN AND ANALYSIS FOR ALGORITHM

Shortest Path Comparison — Dijkstra vs Bellman-Ford

Given Graph

Consider the following directed weighted graph:

- $A \rightarrow B = 4$
- $A \rightarrow C = 2$
- $C \rightarrow B = -3$
- $B \rightarrow D = 2$
- $C \rightarrow D = 5$
- $D \rightarrow E = 1$
- $B \rightarrow E = 7$

Source vertex = A

1. Dijkstra's Algorithm

Steps:

1. Initialize distance of A = 0 and all other vertices = ∞ .
2. Select the unvisited vertex with the smallest distance.
3. Relax its outgoing edges.
4. Repeat until all vertices are processed.

Starting from A:

Vertex Initial Final shortest distance

A 0 0

Vertex Initial Final shortest distance

B ∞ -1

C ∞ 2

D ∞ 1

E ∞ 2

The shortest paths are:

- $A \rightarrow C = 2$
- $A \rightarrow C \rightarrow B = 2 + (-3) = -1$
- $A \rightarrow C \rightarrow B \rightarrow D = -1 + 2 = 1$
- $A \rightarrow C \rightarrow B \rightarrow D \rightarrow E = 1 + 1 = 2$

However, Dijkstra is not guaranteed to produce these correct results when negative edges are present.

2. Why Dijkstra Fails

Dijkstra assumes that once a vertex is selected as having the smallest current distance, its distance can never become smaller.

This assumption is valid only when all edge weights are non-negative.

With a negative edge such as:

$$C \rightarrow B = -3$$

a vertex that was already considered finalized may later receive a smaller distance through another vertex.

Therefore:

Dijkstra cannot reliably handle graphs containing negative-weight edges.

It can sometimes produce the correct answer by chance, but there is no correctness guarantee.

3. Bellman-Ford Algorithm

Bellman-Ford handles both positive and negative edge weights.

Steps

1. Set source distance to 0.
2. Set all other distances to ∞ .
3. Relax every edge $V - 1$ times.
4. Perform one additional iteration.
5. If any distance still decreases, the graph contains a negative-weight cycle.

For our graph:

Initial

Vertex Distance

A	0
B	∞
C	∞
D	∞
E	∞

After relaxation

Vertex Shortest Distance

A 0

B -1

C 2

D 1

E 2

Therefore:

Shortest path from A to E:

$A \rightarrow C \rightarrow B \rightarrow D \rightarrow E$

Cost:

$$2 - 3 + 2 + 1 = 2$$

4. Complexity Comparison

Feature	Dijkstra	Bellman-Ford
Negative edges	Not supported	Supported
Negative cycles	Cannot handle	Can detect
Time complexity	$O((V+E) \log V)$	$O(VE)$
Space complexity	$O(V+E)$	$O(V)$
Speed	Faster	Slower

For a dense graph, Dijkstra can also be implemented in $O(V^2)$.

5. Correctness Analysis

Dijkstra

Dijkstra is correct when:

All edge weights ≥ 0

Because no future path can reduce the distance of an already finalized vertex.

With negative edges, this property breaks.

Bellman-Ford

Bellman-Ford is correct even with negative edges because it repeatedly relaxes all edges. After at most $V - 1$ iterations, every shortest simple path has been considered.

6. Performance Analysis

Dijkstra is significantly faster for large graphs because it does not repeatedly examine every edge.

Bellman-Ford examines all edges repeatedly, making it slower:

Dijkstra: $O((V+E) \log V)$

Bellman-Ford: $O(VE)$

Therefore, when the graph contains only non-negative weights, Dijkstra is generally preferred.

When negative weights are possible, Bellman-Ford is safer despite its higher computational cost.

7. Algorithm Selection Insights

- Use Dijkstra when all edge weights are non-negative and performance is important.
- Use Bellman-Ford when negative edge weights exist.
- Use Bellman-Ford when negative-cycle detection is required.
- For very large graphs with non-negative weights, Dijkstra is usually more efficient.

Conclusion

Dijkstra is faster but limited to graphs with non-negative edge weights. Bellman-Ford is slower but more flexible because it supports negative edges and detects negative-weight cycles. Therefore, algorithm selection should depend primarily on the presence of negative edges and the required performance.